

BOM

Browser Object Model

Complete Theory Guide

window · location · history · navigator · screen · dialogs · cookies ·
Web Storage

window	location	history	navigator	screen	Dialogs & Timers	Cookies	Web Storage
--------	----------	---------	-----------	--------	------------------------	---------	----------------

What is the Browser Object Model (BOM)?

The **Browser Object Model (BOM)** is a set of JavaScript objects provided by the browser that allow scripts to interact with the browser window itself — independently of the web page content. Unlike the DOM (Document Object Model), the BOM has no official W3C standard, but all modern browsers implement it consistently.

The BOM's top-level object is **window**, which represents the browser window or tab. Every global variable, function, and object in JavaScript is actually a property of the window object.

BOM Object Hierarchy

window (Global Object)

window.document · window.location · window.history · window.navigator · window.screen

Key BOM vs DOM Differences

Feature	BOM	DOM
Standard	No official standard	W3C Standard
Focus	Browser window/tab	HTML document content
Root object	window	document
Examples	location, history, navigator	getElementById, querySelector

TOPIC 15 window Object

The Global Browser Object

The **window** object is the global object in a browser environment. Every global variable and function you declare automatically becomes a property of window. It is the outermost container of all BOM objects.

■ **NOTE:** In Node.js the global object is called **global**, not window. Code that relies on window will not run in Node.js.

Important Properties

Property / Method	Type	Description
<code>window.innerWidth</code>	number	Width of the viewport (px), excluding scrollbar
<code>window.innerHeight</code>	number	Height of the viewport (px), excluding scrollbar
<code>window.outerWidth</code>	number	Width of entire browser window including chrome
<code>window.outerHeight</code>	number	Height of entire browser window including chrome
<code>window.scrollX</code>	number	Pixels scrolled horizontally from the left
<code>window.scrollY</code>	number	Pixels scrolled vertically from the top
<code>window.devicePixelRatio</code>	number	Ratio of CSS pixels to physical pixels (2 on Retina)
<code>window.document</code>	object	Reference to the current DOM document
<code>window.location</code>	object	Current URL information and navigation methods
<code>window.history</code>	object	Browser session history object
<code>window.navigator</code>	object	Browser/device information object
<code>window.screen</code>	object	Physical screen information object
<code>window.console</code>	object	Browser developer console

Important Methods

Property / Method	Type	Description
<code>window.open(url, target, specs)</code>	function	Opens a new window/tab. Returns the new window reference.
<code>window.close()</code>	function	Closes the current window (only works on script-opened windows)
<code>window.focus()</code>	function	Brings the window to the front
<code>window.blur()</code>	function	Removes focus from the window
<code>window.scrollTo(x, y)</code>	function	Scrolls to an absolute position in the document

Property / Method	Type	Description
<code>window.scrollTo(x,y)</code>	function	Scrolls by a relative amount from current position
<code>window.resizeTo(w,h)</code>	function	Resizes the window to given width and height
<code>window.print()</code>	function	Opens the browser print dialog

✓ **TIP:** Since window is global, you can omit it: `window.alert()` and `alert()` are identical. Same for `console.log`, `setTimeout`, etc.

TOPIC 16 window.location

URL Information & Navigation

The **location** object contains information about the current URL and provides methods to navigate the browser to a new page, replace the current page in history, or reload the page.

URL Anatomy

`https://example.com:8080/path/page.html?name=JS&ver=2#section`

Property / Method	Type	Description
<code>location.href</code>	string	The full URL string. Setting it navigates to a new URL.
<code>location.protocol</code>	string	The protocol scheme: 'http:' or 'https:'
<code>location.host</code>	string	Hostname + port: 'example.com:8080'
<code>location.hostname</code>	string	Only hostname without port: 'example.com'
<code>location.port</code>	string	Port number as string: '8080' (empty if default)
<code>location.pathname</code>	string	Path after the host: '/path/page.html'
<code>location.search</code>	string	Query string including '?': '?name=JS&ver=2'
<code>location.hash</code>	string	Fragment identifier including '#': '#section'
<code>location.origin</code>	string	Protocol + host: 'https://example.com:8080'

Navigation Methods

Property / Method	Type	Description
<code>location.assign(url)</code>	function	Loads a new URL, adding it to browser history
<code>location.replace(url)</code>	function	Loads a new URL, replacing current in history (no back)
<code>location.reload()</code>	function	Reloads the current page (from cache by default)

✓ **TIP:** Use `location.replace()` for login redirects so the user cannot click Back to return to the login page.

■ **NOTE:** `location.search` returns the raw query string. Use `URLSearchParams` to parse individual parameters easily.

TOPIC 17 window.history

Session History & SPA Navigation

The **history** object lets JavaScript navigate through the browser's session history stack. Modern Single Page Applications (SPAs) use **pushState** and **replaceState** to update the URL without a full page reload.

Properties

Property / Method	Type	Description
<code>history.length</code>	number	Number of entries in the current session's history stack
<code>history.state</code>	any	The state object associated with the current history entry
<code>history.scrollRestoration</code>	string	'auto' or 'manual' – controls scroll position on navigation

Methods

Property / Method	Type	Description
<code>history.back()</code>	function	Go to the previous page (same as clicking browser Back)
<code>history.forward()</code>	function	Go to the next page (same as clicking browser Forward)
<code>history.go(n)</code>	function	Jump n steps: <code>go(-2)</code> = 2 pages back, <code>go(1)</code> = 1 forward
<code>history.pushState(state, title, url)</code>	function	Add a new entry to the history stack and update the URL
<code>history.replaceState(state, title, url)</code>	function	Update current history entry without creating a new one

The History Stack

Each time you visit a page, an entry is pushed onto the history stack. `pushState()` lets SPAs add entries without reloading, enabling bookmarkable URLs.

Action	Method / Event	History Stack Result
Visit /home	browser navigation	[/home] ← current
Visit /about	browser navigation	[/home, /about] ← current
<code>pushState /profile</code>	<code>history.pushState()</code>	[/home, /about, /profile] ← current
<code>back()</code>	<code>history.back()</code>	[/home, /about ← current, /profile]
<code>replaceState /settings</code>	<code>history.replaceState()</code>	[/home, /settings ← current, /profile]

■ **NOTE:** The popstate event fires when the active history entry changes (e.g., user clicks Back/Forward). Listen with `window.addEventListener('popstate', handler)`.

TOPIC 18 window.navigator

Browser & Device Information

The **navigator** object contains information about the browser application and the device running it. It also provides APIs for geolocation, clipboard, media devices, service workers, and more.

Core Properties

Property / Method	Type	Description
<code>navigator.userAgent</code>	string	Full browser user-agent string (can be spoofed)
<code>navigator.language</code>	string	Preferred language of the browser, e.g. 'en-US'
<code>navigator.languages</code>	string[]	Ordered list of user's preferred languages
<code>navigator.platform</code>	string	OS/platform string e.g. 'Win32', 'MacIntel', 'Linux'
<code>navigator.onLine</code>	boolean	true if browser has network access
<code>navigator.cookieEnabled</code>	boolean	true if cookies are enabled
<code>navigator.hardwareConcurrency</code>	number	Number of logical CPU cores available
<code>navigator.maxTouchPoints</code>	number	Maximum number of simultaneous touch points
<code>navigator.vendor</code>	string	Browser vendor name e.g. 'Google Inc.'

Key APIs

Property / Method	Type	Description
<code>navigator.geolocation</code>	object	Access to Geolocation API (<code>getCurrentPosition</code> , <code>watchPosition</code>)
<code>navigator.clipboard</code>	object	Async Clipboard API (<code>writeText</code> , <code>readText</code> , <code>write</code> , <code>read</code>)
<code>navigator.mediaDevices</code>	object	Access to camera, microphone via <code>getUserMedia()</code>
<code>navigator.serviceWorker</code>	object	Register/manage Service Workers for PWAs
<code>navigator.permissions</code>	object	Query/request permissions (camera, notifications, etc.)
<code>navigator.sendBeacon(url, data)</code>	function	Send analytics data reliably on page unload
<code>navigator.vibrate(pattern)</code>	function	Vibrate the device (mobile only)
<code>navigator.share(data)</code>	function	Native share dialog on supported mobile browsers

■ **WARNING:** `navigator.userAgent` is unreliable for browser detection — browsers often spoof it. Prefer feature detection: `if ('geolocation' in navigator) { ... }`

■ **NOTE:** Geolocation and Clipboard require user permission. The browser shows a permission prompt the first time.

TOPIC 19 window.screen

Physical Display Information

The **screen** object provides information about the user's physical display screen. Unlike `window.innerWidth` (viewport), `screen.width` refers to the actual monitor resolution.

Properties

Property / Method	Type	Description
<code>screen.width</code>	number	Total width of the screen in CSS pixels
<code>screen.height</code>	number	Total height of the screen in CSS pixels
<code>screen.availWidth</code>	number	Width available to browser (excludes taskbar, dock)
<code>screen.availHeight</code>	number	Height available to browser (excludes taskbar, dock)
<code>screen.availLeft</code>	number	X coordinate of the first available pixel on screen
<code>screen.availTop</code>	number	Y coordinate of the first available pixel on screen
<code>screen.colorDepth</code>	number	Bits used per pixel for color: usually 24
<code>screen.pixelDepth</code>	number	Bits per pixel of the display (usually same as <code>colorDepth</code>)
<code>screen.orientation</code>	object	ScreenOrientation object: type ('landscape-primary' etc.) and angle

screen vs window Size Comparison

Property	What it measures	Typical value
<code>screen.width</code>	Physical screen width	1920px
<code>screen.availWidth</code>	Screen width minus OS chrome	1920px
<code>window.outerWidth</code>	Browser window total width	1366px
<code>window.innerWidth</code>	Viewport width (content area)	1349px
<code>document.body.clientWidth</code>	Content width minus scrollbar	1332px

✓ **TIP:** `window.devicePixelRatio` tells you whether the screen is high-DPI (Retina). A value of 2 means 2 physical pixels per CSS pixel.

TOPIC 20 Dialogs & Timers

User Prompts and Delayed Execution

The BOM provides three blocking dialog boxes — **alert**, **confirm**, and **prompt** — and four timer functions — **setTimeout**, **setInterval**, **clearTimeout**, and **clearInterval** — for scheduling code execution.

Dialog Methods

Property / Method	Type	Description
<code>alert(message)</code>	<code>void</code>	Shows a message box. Execution pauses until OK is clicked.
<code>confirm(message)</code>	<code>boolean</code>	Shows OK/Cancel dialog. Returns true (OK) or false (Cancel).
<code>prompt(message, default)</code>	<code>string null</code>	Shows an input dialog. Returns the string entered or null.

■ **WARNING:** `alert()`, `confirm()`, and `prompt()` block script execution. Avoid them in production — use custom modal dialogs instead.

Timer Functions

Property / Method	Type	Description
<code>setTimeout(fn, ms)</code>	<code>number</code>	Run <code>fn</code> once after <code>ms</code> milliseconds. Returns a timer ID.
<code>clearTimeout(id)</code>	<code>void</code>	Cancel a pending <code>setTimeout</code> by its ID.
<code>setInterval(fn, ms)</code>	<code>number</code>	Run <code>fn</code> every <code>ms</code> milliseconds. Returns a timer ID.
<code>clearInterval(id)</code>	<code>void</code>	Stop a running <code>setInterval</code> by its ID.
<code>requestAnimationFrame(fn)</code>	<code>number</code>	Run <code>fn</code> before the next repaint (~60fps). Best for animations.
<code>cancelAnimationFrame(id)</code>	<code>void</code>	Cancel a pending <code>requestAnimationFrame</code> call.

How the Event Loop Handles Timers

JavaScript is single-threaded. Timer callbacks are placed in the **macrotask queue** after the specified delay, and executed only when the call stack is empty. This means a `setTimeout(fn, 0)` does not run immediately — it runs after the current synchronous code finishes.

✓ **TIP:** `setInterval` can drift over time. For precise intervals, use recursive `setTimeout` instead.

TOPIC 21 Cookies

Persistent Client-Side Data Storage

Cookies are small pieces of data (max ~4KB each) stored by the browser and automatically sent to the server with every matching HTTP request. They are primarily used for session management, personalisation, and tracking.

Cookie Attributes

Property / Method	Type	Description
<code>name=value</code>	required	The key-value pair of the cookie
<code>expires=date</code>	optional	Expiry date in UTC format. If omitted, cookie is session-only.
<code>max-age=seconds</code>	optional	Lifetime in seconds from now. Overrides expires if both given.
<code>path=/</code>	optional	URL path scope. '/' means the whole domain.
<code>domain=example.com</code>	optional	Which domain(s) can receive the cookie.
<code>secure</code>	optional	Cookie sent only over HTTPS.
<code>HttpOnly</code>	optional	Cookie inaccessible to JavaScript (set by server only).
<code>SameSite=Strict Lax None</code>	optional	Controls cross-site request behaviour (CSRF protection).

Cookie Lifecycle

Type	When Set	When Deleted
Session cookie	No expires/max-age	When browser tab/window is closed
Persistent cookie	expires or max-age set	At the specified date/time
Deleted cookie	max-age=0 or past date	Immediately by the browser

■ **WARNING:** Cookies marked `HttpOnly` cannot be read or modified by JavaScript — only by the server. This protects against XSS attacks.

✓ **TIP:** Modern alternatives to cookies for client-only storage are `localStorage` and `sessionStorage`, which don't get sent to the server.

TOPIC 22 Web Storage

localStorage & sessionStorage

The **Web Storage API** provides two storage objects — **localStorage** and **sessionStorage** — that store key-value pairs in the browser. Both offer ~5–10MB of storage per origin and are never sent to the server.

localStorage vs sessionStorage

Feature	localStorage	sessionStorage
Lifetime	Until manually cleared	Until tab/window is closed
Scope	All tabs of the same origin	Same tab only
Capacity	~5–10 MB	~5–10 MB
Server access	No	No
Sent with requests	No	No
Use case	User preferences, theme, cart	Temp session data, form state

Storage API Methods

Property / Method	Type	Description
<code>storage.setItem(key, value)</code>	void	Store a value. Both key and value must be strings.
<code>storage.getItem(key)</code>	string null	Retrieve a value. Returns null if key not found.
<code>storage.removeItem(key)</code>	void	Delete a specific key-value pair.
<code>storage.clear()</code>	void	Remove all key-value pairs from storage.
<code>storage.key(index)</code>	string null	Returns the name of the nth key.
<code>storage.length</code>	number	Number of items currently stored.

The storage Event

When localStorage is modified in one tab, a **storage** event fires in all other tabs of the same origin. This enables cross-tab communication without a server.

■ **WARNING:** Web Storage only stores strings. Always use `JSON.stringify()` before saving objects and `JSON.parse()` when reading them back.

✓ **TIP:** Web Storage is synchronous and blocks the main thread. For large data, consider IndexedDB which is asynchronous.

Storage Comparison: Cookies vs localStorage vs sessionStorage

Feature	Cookies	localStorage	sessionStorage
---------	---------	--------------	----------------

Capacity	~4 KB	~5–10 MB	~5–10 MB
Expiry	Manually set	Never expires	Tab close
Sent to server	Yes	No	No
Accessible via JS	Yes*	Yes	Yes
Cross-tab	Yes	Yes	No
API complexity	Manual parsing	Simple	Simple

** Cookies with HttpOnly flag cannot be accessed by JavaScript.*